Foundation Models for Tabular Data

From trees to TFMs, and on to causal effects

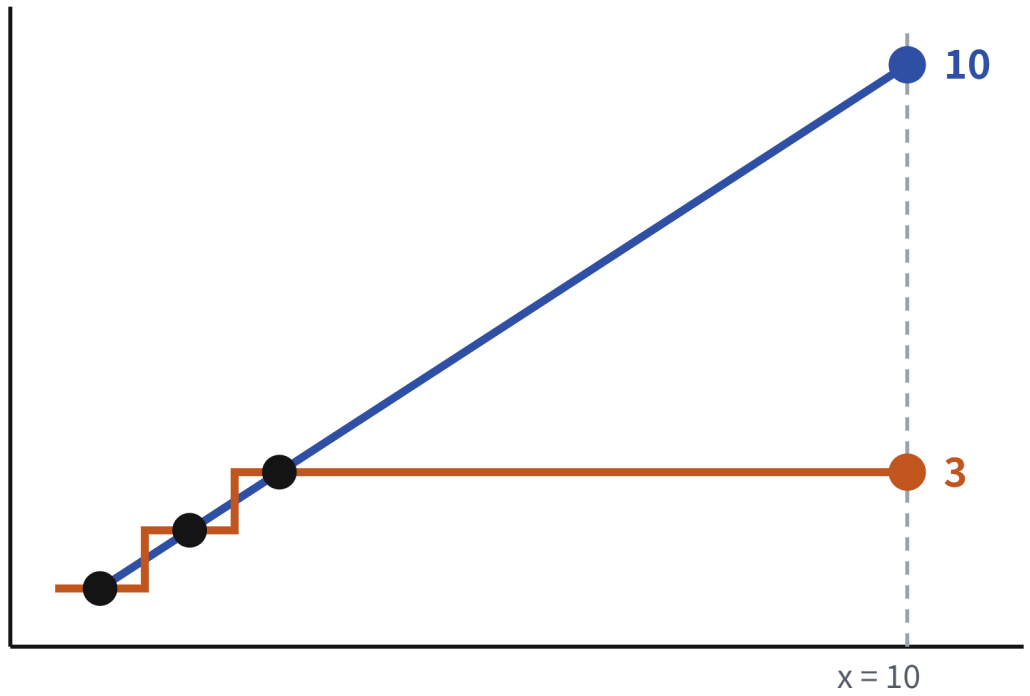
Woohyun Song

Dept. of Computer Science and Engineering, Seoul National University

Today's questions

1. Why have trees beaten deep learning on tabular data?
2. How did TFMs get past that limit, and why are they taking off now?
3. What new problem appears when the same engine estimates causal effects?

Inductive bias: every model fills the gaps with assumptions



- Training set: (1, 1), (2, 2), (3, 3). Both models fit it with zero error
- Linear regression: 10 at x = 10
- Decision tree: 3 at x = 10
- Which is right is decided by the world that generated the data

Definition

Inductive bias: a model's preference for one hypothesis over another

Trees ask one-column questions, one after another

Example: daily rentals at one bike-share station (toy data)

Rainfall > 5 mm? → yes: **20 rentals**

no → Weekend? → yes: **120**, no: **80**

Candidate columns: rainfall, weekend, staff ID (a useless column)

- A useless column (staff ID) rarely reduces the error, so it is rarely picked
- A sharp threshold between 4.9 mm and 5.1 mm takes a single question
- Only the order of values matters, so monotone transforms (rescaling, log) change nothing
- XGBoost and LightGBM boost hundreds of such trees and inherit the same traits

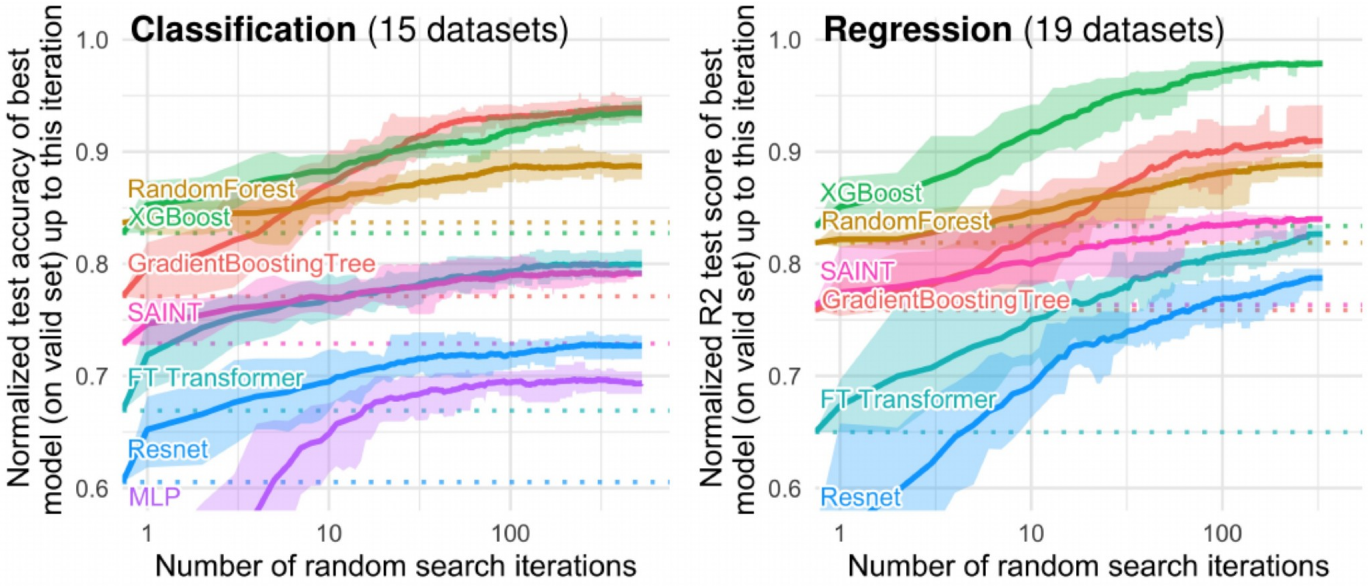
An MLP mixes every column

One neuron in the first layer

$$h = \mathrm{ReLU}\big(w_1 \cdot \mathrm{rain} + w_2 \cdot \mathrm{weekend} + w_3 \cdot \mathrm{staff\ ID} + b\big)$$

- To ignore a useless column, w_3 must become exactly 0, which is hard with little data
- Sensitive to scale → standardization (mean 0, standard deviation 1) is a must
- Standardization fixes scale only; the useless column still leaks in

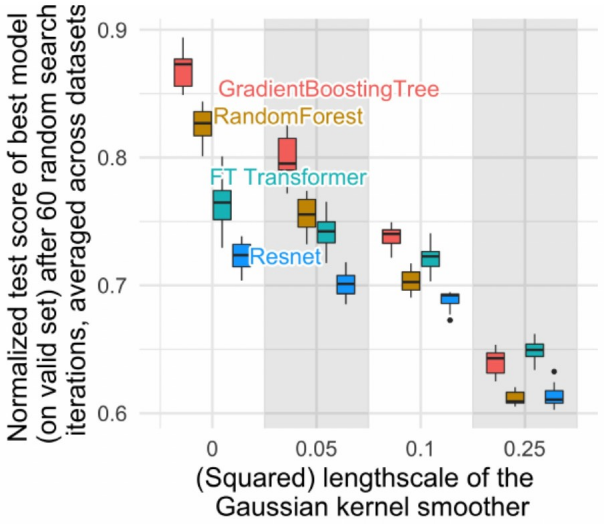
Grinsztajn et al. (NeurIPS 2022): more tuning, trees still win



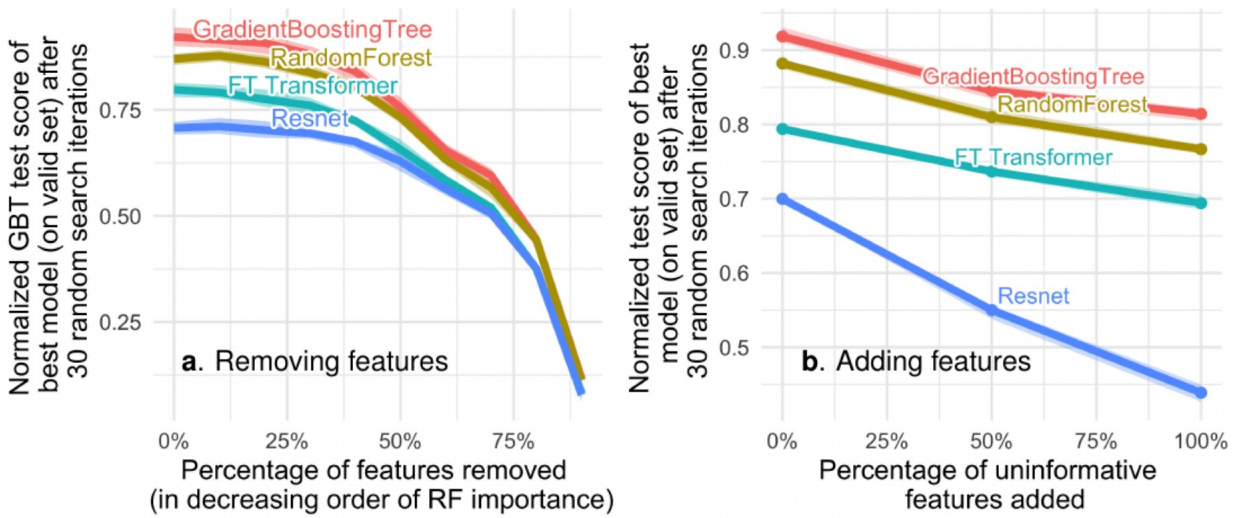
x: random-search iterations, y: normalized score · Source: Grinsztajn et al. (2022), Fig. 1 (numerical features)

- 45 datasets (training set truncated to 10,000 rows), ~400 random-search iterations per dataset
- At every search budget, the best tree model (RF, GBT, XGBoost) beats the best neural net (MLP, ResNet, FT-Transformer, SAINT)
- Scope: curated, medium-sized i.i.d. tables (no missing data, balanced classes, too-easy datasets removed)

Changing the data on purpose to isolate the reasons



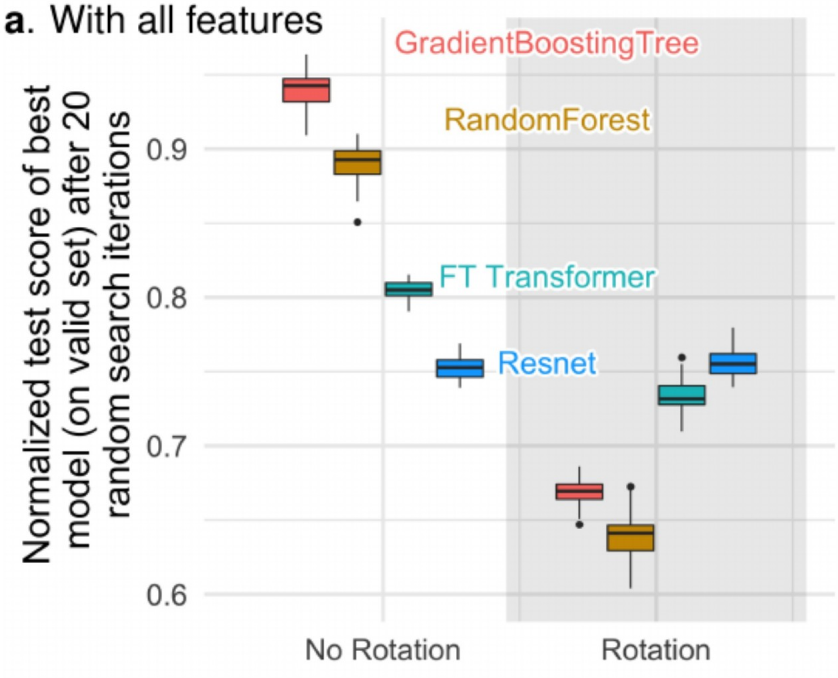
(A) Smoothing the target · Fig. 3



(B) Removing and adding uninformative features · Fig. 5

- (A) Slight smoothing hurts trees far more than neural nets → neural nets were never using the irregular patterns
- (B) Adding random columns hurts ResNet the most → MLP-like nets are fragile to useless features

Rotate the data, and the ranking flips



Source: Grinsztajn et al. (2022), Fig. 6a

- Rotation = replace the columns with mixtures of columns (no information lost)
- Trees drop sharply, ResNet stays the same → the ranking flips
- The original column directions (age, weight, …) carry information
- A rotation-invariant MLP throws this hint away and, in the worst case, needs more data as useless features grow (Ng, 2004)

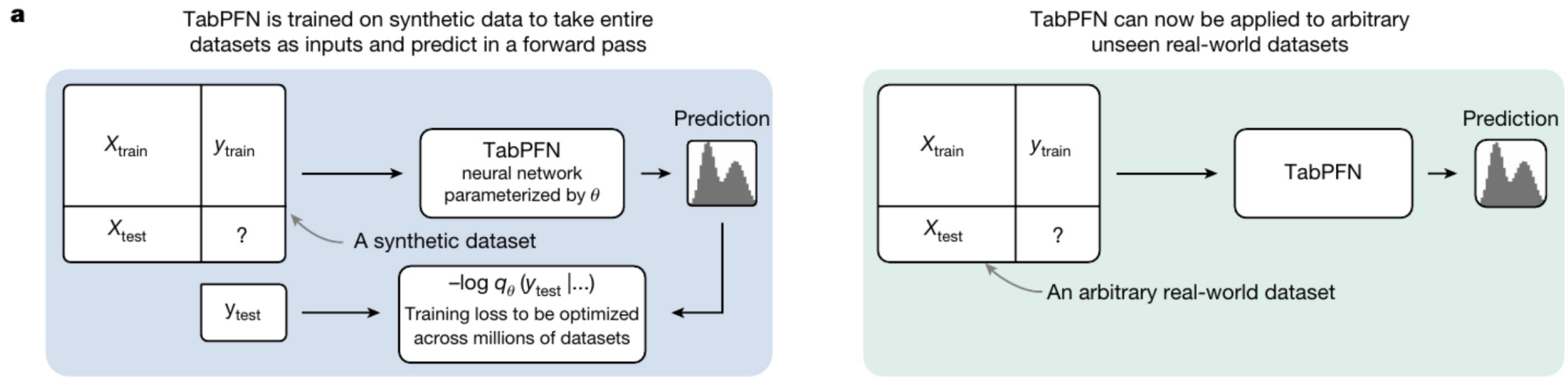
Summary: the trees' strengths are the MLP's biases, inverted

Property of tabular data	Trees	MLPs
Irregular relations (thresholds)	One question draws it	Prefer smooth functions
Many useless columns	Rarely pick them	Mix every column; noise leaks in
Each column has its own meaning	Ask about one column at a time	Rotation-invariant; ignore column directions

Next question

Both models hard-code their bias in the algorithm. Can we set the bias with **data** instead of code?

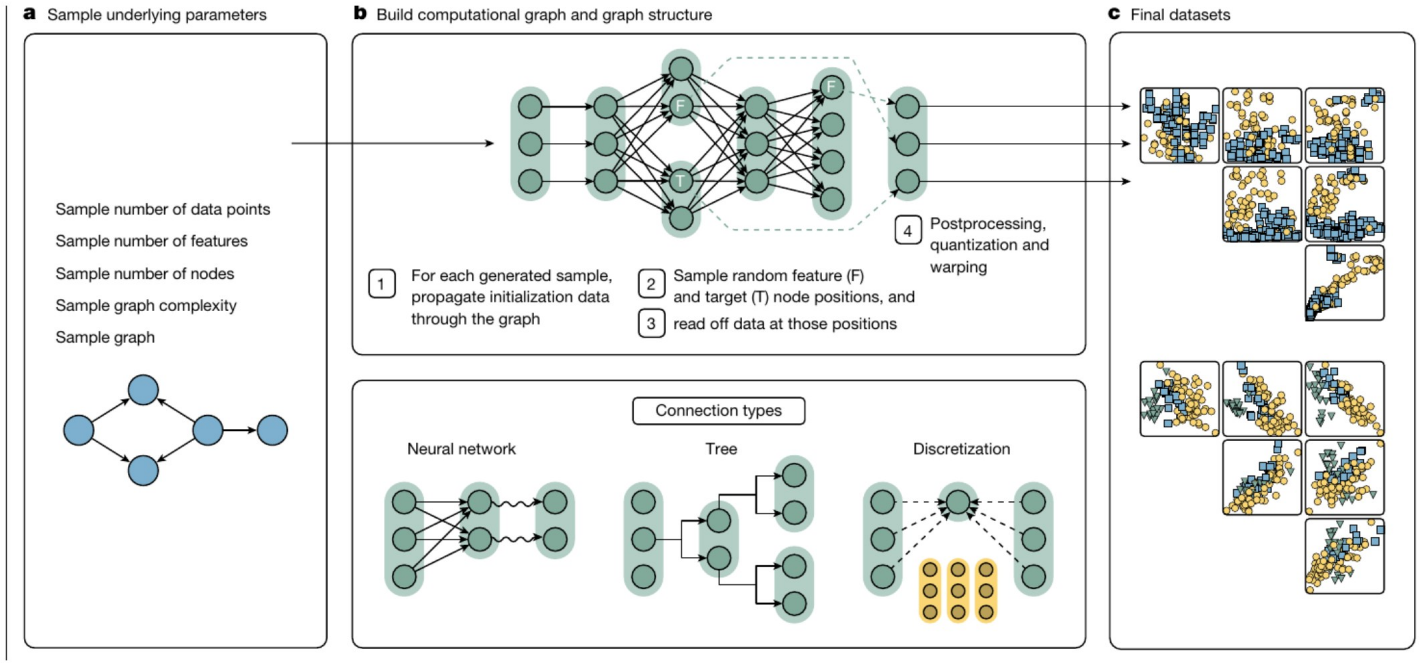
PFNs: set the bias with synthetic data, not code



Source: Hollmann et al., Nature 637 (2025), Fig. 1a

- Pretraining: train a transformer **once** to predict masked targets across synthetic tables
- Use: feed the whole training set as input and predict with forward passes only, weights frozen (in-context learning)
- TabPFN v2: ~130 million synthetic datasets, ~2 weeks on 8 RTX 2080 Ti GPUs

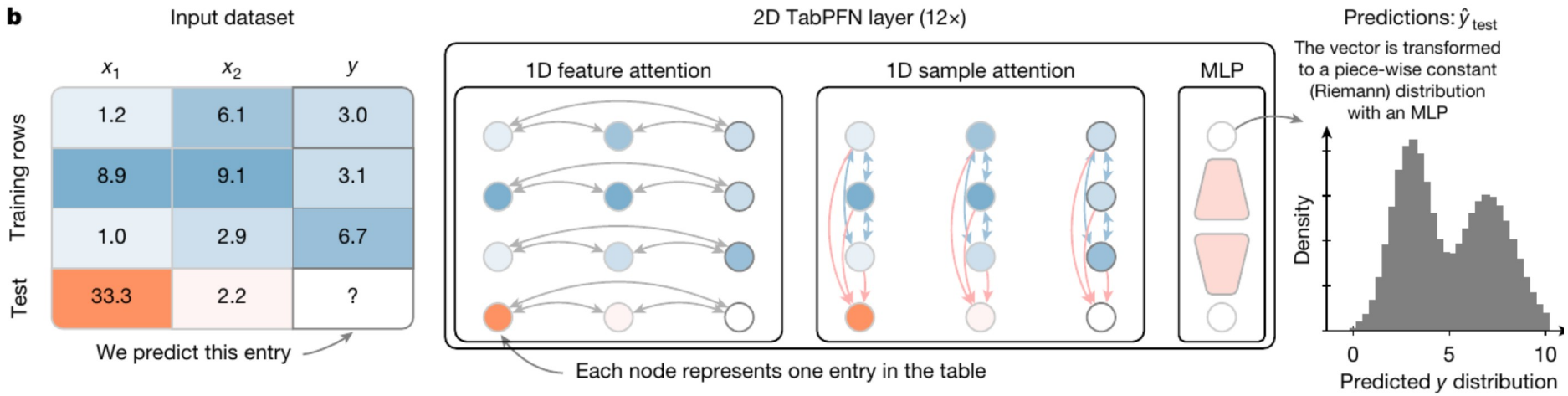
The synthetic data come from causal graphs (SCMs)



Source: Hollmann et al. (2025), Fig. 2

- SCM: each variable = a function of its parents + noise; each edge is a small neural net, a decision tree, or a discretization
- Features from graph pieces cut off from the target → naturally useless columns
- Still, TabPFN learns prediction, not the effect of an intervention → Part 3

One vector per cell, attention along rows and columns



Source: Hollmann et al. (2025), Fig. 1b

- Every cell becomes a vector → each column keeps its identity throughout
- Attention across features in a row, then across training rows in a column, repeated 12 times
- Test rows attend only to training rows; regression outputs a binned distribution, not a single value

Where there is no data, the prior answers

What a PFN approximates (Müller et al., 2022)

$$p(y \mid x^*, D) \;\propto\; \int \underbrace{p(y \mid x^*, \phi)}_{\text{prediction under } \phi} \; \underbrace{p(D \mid \phi)}_{\text{likelihood}} \; \underbrace{p(\phi)}_{\text{prior}} \, d\phi$$

ϕ: one data-generating rule (an SCM) · D: the training set · x^*: a new input · $p(\phi)$: how likely each SCM is to be drawn

Example: the three points from Part 1, prior = 70% linear world + 30% flat world

Both worlds explain the three points perfectly → beliefs stay at 70 : 30

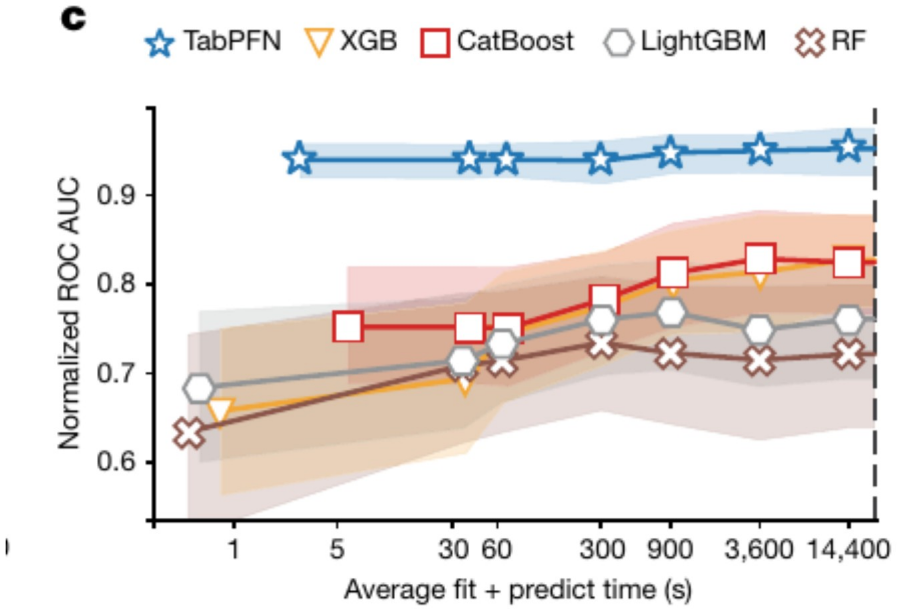
Prediction at x = 10: two peaks, near 10 (70%) and near 3 (30%)

Mean = 0.7 × 10 + 0.3 × 3 = 7.9, a value neither world produces → look at quantiles, not just the mean

How TFMs answer the three weaknesses from Part 1

MLP weakness	TFM response
Useless columns	v1 was hurt by them; v2 puts such columns in the prior → barely affected when uninformative columns are added
Bias toward smoothness	Decision trees and step functions in the prior; TabICL fills 30% of its prior with tree-based SCMs
Ignoring column direction	v1 (one token per row) was relatively rotation-invariant; v2 represents each cell separately, with per-column embeddings

Results: 2.8 seconds beats 4 hours of tuning



Classification · leftmost point of each curve = default settings · Source: Hollmann et al. (2025), Fig. 4c

- 29 classification + 28 regression datasets ($\leq$ 10,000 rows, $\leq$ 500 features)
- Default TabPFN (2.8 s on average) > CatBoost, XGBoost, LightGBM, RF tuned for 4 hours
- Also ahead on the Grinsztajn et al. benchmark, where trees were strong (Extended Data Fig. 2)
- Caveat: the plot shows normalized scores. Raw mean ROC AUC is 0.929 (TabPFN) vs. 0.920 (tuned CatBoost)

Why now?

2022 PFN idea (Müller et al.) · TabPFN v1 (arXiv 2022, ICLR 2023): $\leq$ 1,000 rows, $\leq$ 100 numeric features, classification only

2023 McElfresh et al. compare 19 algorithms on 176 datasets: TabPFN v1 has the best average rank on a 98-dataset subset

2025 Nature version: 10,000 rows, 500 features, regression, categorical and missing values · TabArena: first by a large margin on the 33 datasets within its limits

2026 TabICLv2, TabPFN-3 (1M rows), Google TabFM, TabPFN-3.5 · mostly self-reported

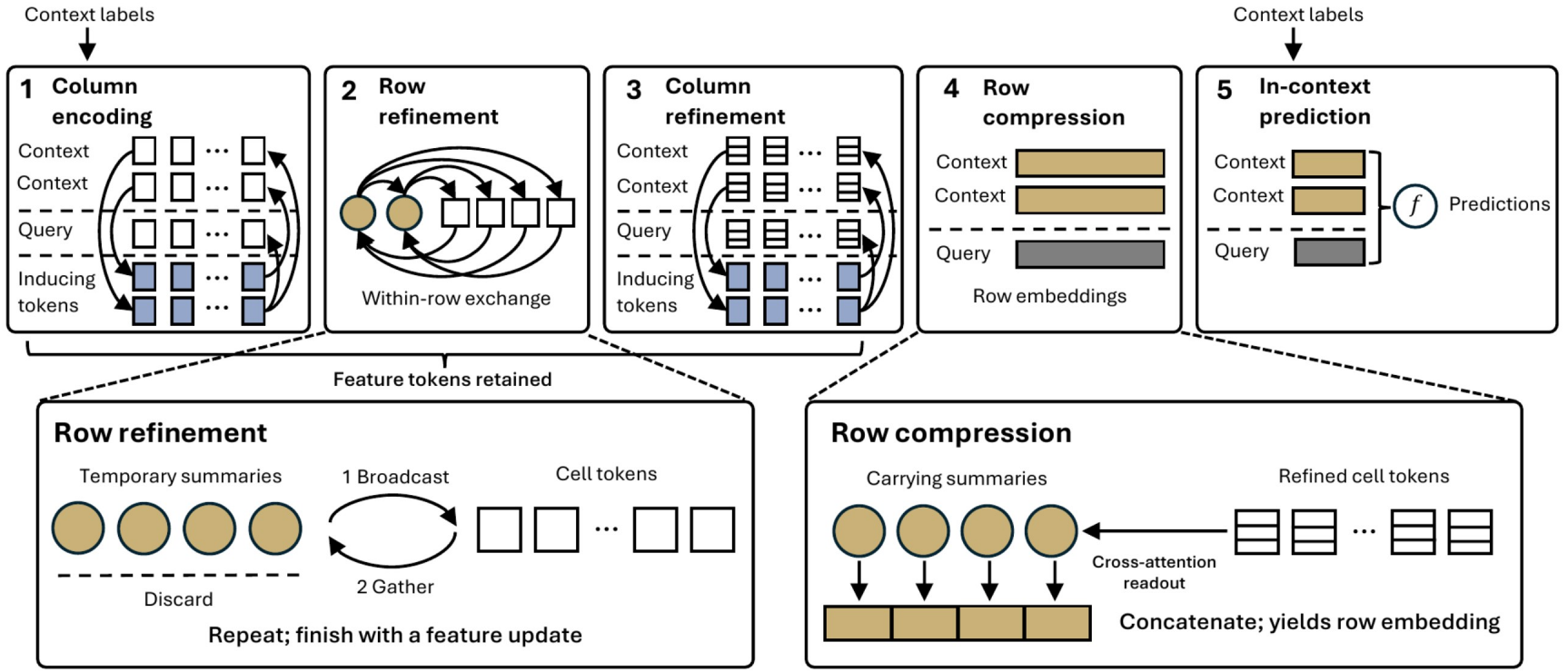
- Molnar: "I have yet to see an XGBoost startup", yet TFM startups exist (Prior Labs, Neuralk-AI, …), and Google, SAP and LG ship their own models (Ch. 11)

Causilo (Nums AI, 2026): refine first, compress later

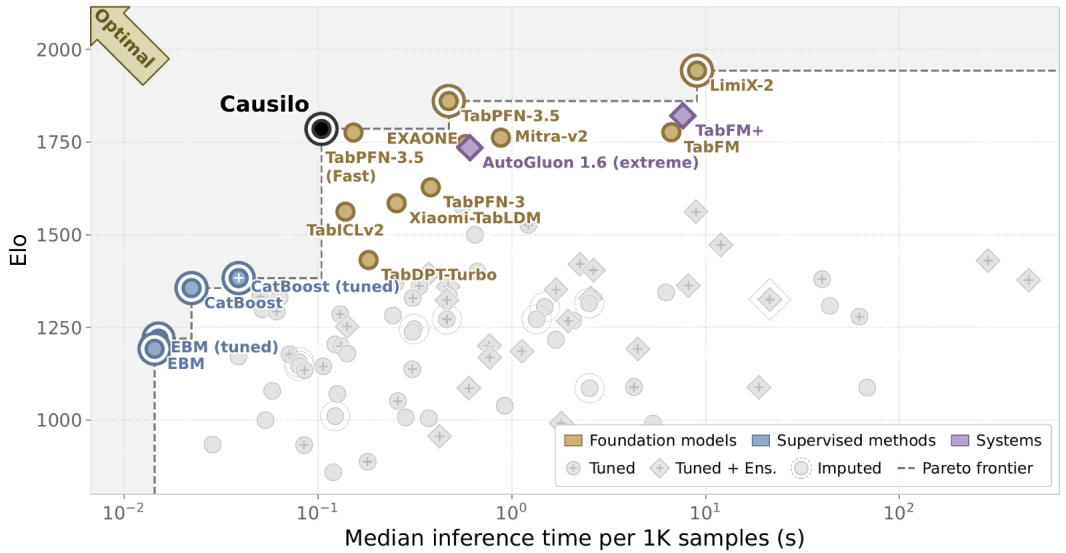
Keep cell tokens throughout
TabPFN-2.5, LimiX, EXAONE-Tabular · every layer updates every cell, within rows and across rows

Compress each row early
TabICL, TabICLv2, TabPFN-3 · each row becomes one vector, then in-context learning over rows

Causilo
Compresses rows too, but first refines cells once more within the row and across rows



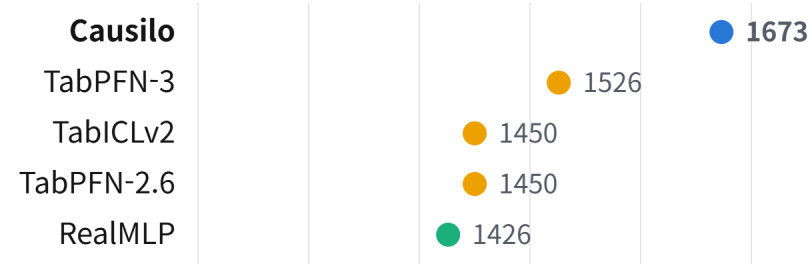
Causilo: fast on TabArena, weaker on unseen groups



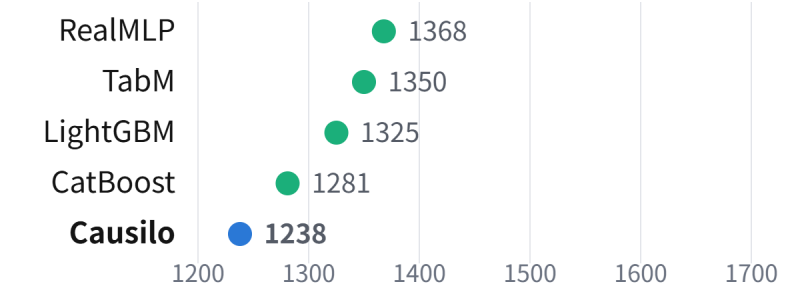
TabArena (51 datasets) · Cho et al. (2026), Fig. 1

BeyondArena regression Elo, top 5 per split

IID split (26 datasets)



Grouped split (7 datasets)



Causilo · Other TFMs · Trees & MLPs

Classification: 1st on IID (77 datasets), 4th on grouped (11) · Cho et al. (2026), Figs. 7–8

Where TFMs still fall short

- **Beyond IID**: TFMs excel on tiny- to medium-sized IID data; trees and other deep models still lead on non-IID, large and high-dimensional data (BeyondArena: 142 datasets, 11 models)
- **Enterprise data**: on SAP's internal EGI-Bench (557 tasks), even an untuned random forest beat TabPFN-2.5 and TabICLv2 (Kim et al., 2026)
- **Real-time prediction**: one sample with 10,000 context rows took 0.2 s for TabPFN v2 (GPU) vs. 0.0002 s for CatBoost (Hollmann et al., 2025)

Prediction and causal effects are different questions

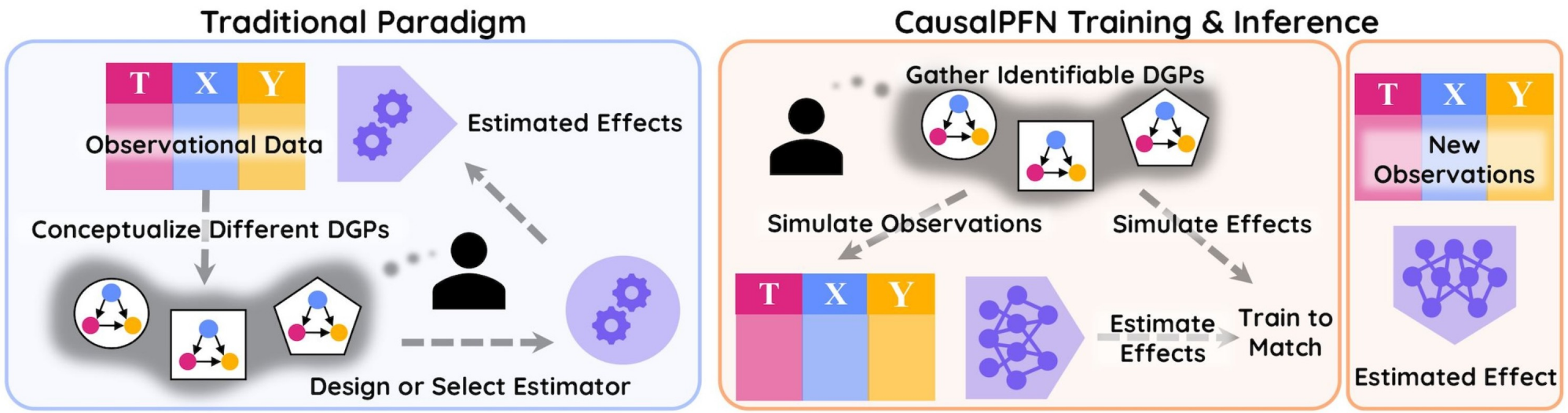
Definition (conditional average treatment effect, CATE)

$$\tau(c) := \mathbb{E}\big[\, Y(1) - Y(0) \mid C = c \,\big]$$

Among units with covariates c: mean outcome difference between the treated and untreated worlds

- Prediction: “What will this student score?”
  Causal: “How much would tutoring change this student’s score?”
- Each person lives in only one of the two worlds → the answer is never directly in the data
- Confounder: affects both treatment and outcome (e.g., prior ability)

Causal foundation models learn effects from simulated worlds

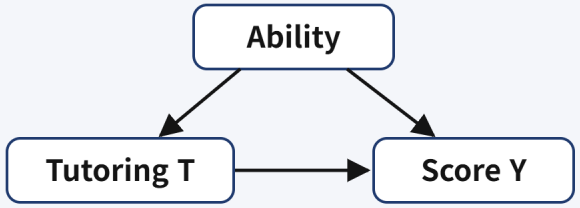


Source: Balazadeh et al. (2025), CausalPFN

- Simulate: sample an SCM → observational rows (T, X, Y), plus the effect from do($T = 1$) and do($T = 0$)
- Train: predict the effect from the observational rows alone → at use time, a real table in, CATE out, no retraining
- Effects are identifiable only if confounders are columns and both groups exist at each X: CausalPFN's prior keeps only such worlds; Do-PFN's prior also has hidden confounders

Making the target: intervene in the simulated world

Observed world

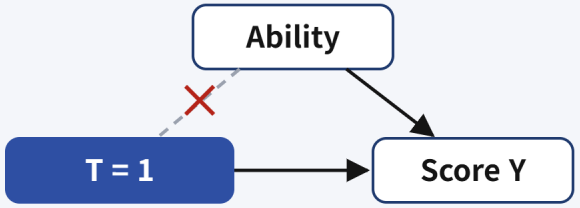


T: lower ability → more often tutored
Y = base score + 5 · T

Student	T	Y
A (high, base 80)	0	**80**
C (low, base 60)	1	**65**

↑ **Model input (context)**

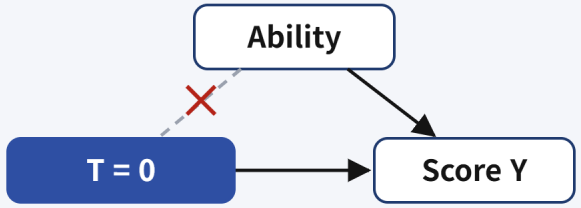
Intervene: do(T = 1)



T's rule removed: T = 1 for all
Y = base score + 5 · T

Student	T	Y
A (high, base 80)	1	**85**
C (low, base 60)	1	**65**

Intervene: do(T = 0)



T's rule removed: T = 0 for all
Y = base score + 5 · T

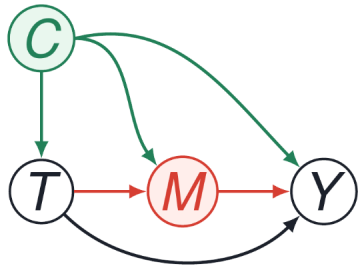
Student	T	Y
A (high, base 80)	0	**80**
C (low, base 60)	0	**60**

Effect: A 85 − 80 = +5, C 65 − 60 = +5 → training target

Dashed boxes: outcomes never seen in real data. Real data holds only the observed world; the other two exist only in simulation.

The trap: same row, asked twice with only the treatment flipped

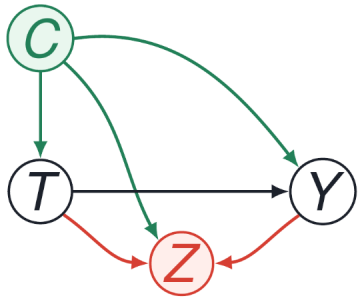
Mediator conditioning



fixing M **erases** the real effect

$\Rightarrow \; -\text{NIE}_Z$

Collider conditioning



conditioning on Z **opens** a spurious link

$\Rightarrow \; +\Delta_{\text{sel}}$

C = pre-treatment

M/Z = post-treatment

Correct query: keep pre-treatment C; drop post-treatment M/Z.

Source: Ham et al. (2026), poster of this paper

- What the CFM computes, Eq. (1):

$$\hat{\tau}_{\text{CFM}}(c, z_{\text{obs}}) := \mathbb{E}\big[Y(1) \mid Z(1) = z_{\text{obs}}, C = c\big] - \mathbb{E}\big[Y(0) \mid Z(0) = z_{\text{obs}}, C = c\big]$$

- Two predictions on the same row, only T flipped: a post-treatment Z in the row stays at $z$obs

Example: tutoring $T \rightarrow$ study hours $M \rightarrow$ score Y

$$M = \delta T + \cdots, \quad Y = \alpha T + \beta M + \cdots$$

$$\text{True effect} = \alpha + \beta\delta = 2 + 3 \times 1 = 5$$

$$\text{CFM answer} = \alpha = 2$$

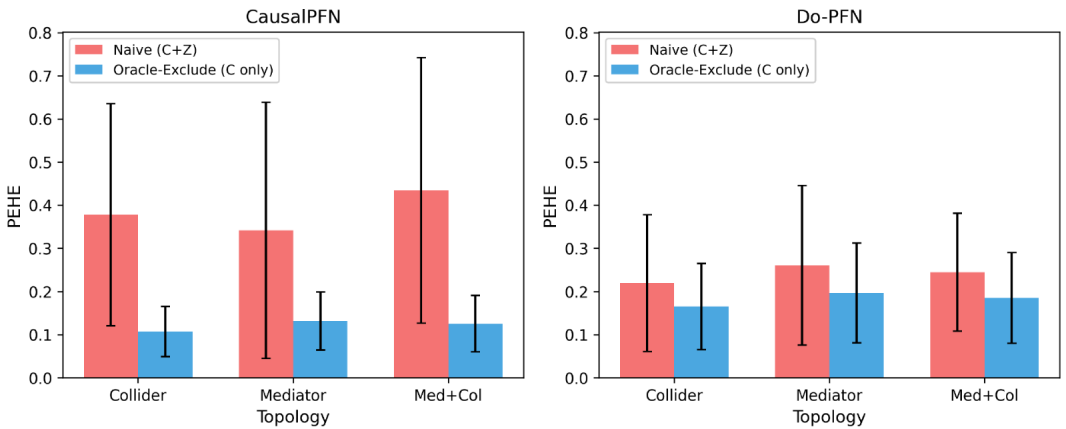
$$\text{Bias} = -\beta\delta = -3$$

Bias decomposition, and the gain from dropping columns

Proposition 3.1 (bias decomposition)

$$\mathrm{Bias}_{\mathrm{CFM}} = -\,\mathrm{NIE}_Z(c) - \Delta_{\mathrm{int}} + \Delta_{\mathrm{sel}}$$

Additive mediator: bias = −NIE · Additive collider: bias = Δsel · Holds even for an optimal-capacity model



Red: all columns (C + Z) · Blue: post-treatment removed (C only)
Error bars: SD across settings · Source: Ham et al. (2026), Fig. 2

- Dropping the true post-treatment columns cuts PEHE: CausalPFN 61.5–71.6%, Do-PFN 24.2–24.7% (Appendix Table 3)
- Dropping as many random pre-treatment columns instead: no gain in any of the 6 cases (−26.8% to +0.2%) → which columns, not how many
- PEHE: error between estimated and true CATE (lower is better)

Without the true graph: TC-LD

1. **Screen**: keep as candidates the columns related to T, or related to Y even after controlling for T
2. **Refine**: regress each candidate on up to 3 other candidates; if the residual becomes unrelated to T, keep it as pre-treatment, otherwise mark it post-treatment
3. **Query**: drop the columns that look post-treatment, then ask the CFM

- Recovers 86.7% of the oracle's gain on average (PEHE: all columns 0.236 → TC-LD 0.123, oracle 0.105). The best classical method (GES) recovers less than half
- Semi-synthetic IHDP and ACIC 2016 with injected mediators: caught all of them; pre-treatment columns wrongly dropped 0% and about 1%
- Limits: worse than dropping nothing in 22.4% of settings; can drop direct causes of T; in a mediator chain, usually catches only the first

Summary

Part 1 Useless columns threw neural nets off

Part 2 TFMs were pretrained to withstand such columns

Part 3 Post-treatment variables can predict the outcome well, yet fixing them biases the effect

Takeaway

Columns that help prediction are not the same as columns safe for effect estimation

Open questions: Do-PFN was trained with descendants of the treatment in its inputs, so why does bias remain? · How do we combine ‘when was it measured?’ with a statistical filter?

References

Grinsztajn, Oyallon, Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? NeurIPS 2022.

Hollmann et al. Accurate predictions on small data with a tabular foundation model. Nature 637, 319–326, 2025.

Müller et al. Transformers can do Bayesian inference. ICLR 2022.

Hollmann et al. TabPFN: A transformer that solves small tabular classification problems in a second. ICLR 2023.

Molnar. Tabular Foundation Models: A short and opinionated guide. 2026.

McElfresh et al. When do neural nets outperform boosted trees on tabular data? arXiv 2305.02997.

Erickson et al. TabArena: A living benchmark for machine learning on tabular data. arXiv 2506.16791.

Purucker et al. Beyond IID: How general are tabular foundation models, really? arXiv 2606.30410.

Kim, Schambach, Essenberger, Sres, Höhne. Exploring differences between tabular enterprise data and public benchmarks. ICML 2026 Workshop on Foundation Models for Structured Data.

Cho, Jeong, Lee, Lee, Yoo. Causilo technical report. arXiv 2609.22866, 2026.

Ham, Kim, Kim, Kim, Lee. Causal foundation models perform better without post-treatment variables. ICML 2026 Workshop on Foundation Models for Structured Data (paper and poster).

Qu, Holzmüller, Varoquaux, Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. arXiv 2502.05564.

Robertson et al. Do-PFN: In-context learning for causal effect estimation. NeurIPS 2025.

Balazadeh et al. CausalPFN: Amortized causal effect estimation via in-context learning. NeurIPS 2025.

Ng. Feature selection, L1 vs. L2 regularization, and rotational invariance. ICML 2004.